Cloud Engineering Portfolio

Dean Eseddi

AWS Hybrid Cloud Infrastructure

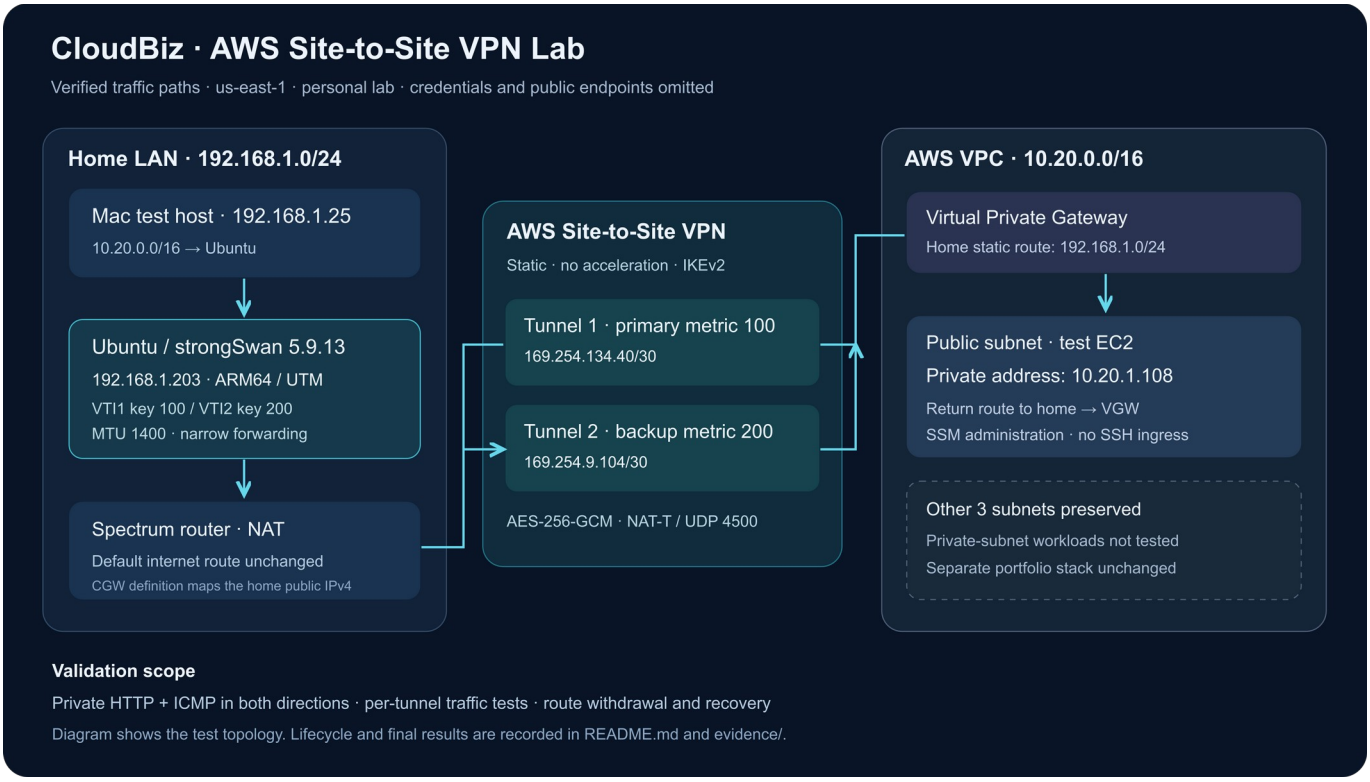
Site-to-Site IPsec VPN | Independent lab | us-east-1 | October 2026

Project overview

Designed and implemented a hybrid cloud networking lab connecting a simulated small-business office network to an AWS VPC through a dual-tunnel Site-to-Site VPN. Both tunnels passed actual private traffic tests. Controlled failover and recovery worked with a measured interruption. The temporary VPN was deleted after validation to control costs.

Business problem and solution

A small business needs secure private access from its office to a cloud-hosted application. The lab models that requirement using an Ubuntu VPN router behind home NAT and an AWS Virtual Private Gateway, with static routing and narrowly scoped test-host access. It is a technical simulation, not a client deployment.



Topology during testing. The EC2 private address is in a public subnet; private-subnet workloads were not tested.

Technologies used

AWS VPC, EC2, Internet Gateway, VGW, CGW, Site-to-Site VPN, route tables, Security Groups, NACL review, CloudWatch metrics, Systems Manager, encrypted gp3 EBS; Ubuntu 24.04 ARM64, strongSwan 5.9.13, UTM, IKEv2, AES-256-GCM, NAT-T, Linux VTIs, UFW, and Apache on Amazon Linux.

Implementation and security

Implementation steps

- Built the VPC at 10.20.0.0/16 with two public and two private subnets across two Availability Zones; configured an Internet Gateway and subnet route-table associations.
- Hosted the CloudBiz demonstration website on the existing Amazon Linux EC2 instance at private IP 10.20.1.108 with Apache and an encrypted 8 GiB gp3 EBS volume.
- Configured a CGW matching the home public IP and a VGW attached to the project VPC; created one standard, non-accelerated static VPN with two tunnels.
- Configured Ubuntu strongSwan with fresh per-tunnel PSKs, IKEv2, AES-GCM and NAT-T; added two keyed VTI interfaces with MTU 1400 and MSS 1360.
- Added the home return route only to the test workload route table and a project route on the designated Mac; used CHILD-SA hooks for route installation and withdrawal.
- Ran independent HTTP observations and AWS-origin tests during each controlled tunnel shutdown and recovery; collected SA counters, logs, route snapshots and native AWS metrics.

Security architecture

The forwarding guard allowed only the designated Mac/EC2 pair for HTTP and ICMP through either VTI. Incoming IKE/NAT-T was restricted to AWS tunnel endpoints; a temporary proof HTTP listener accepted only the EC2 private source. The high-metric blackhole prevented fallback to the home default route when both VPN routes were absent.

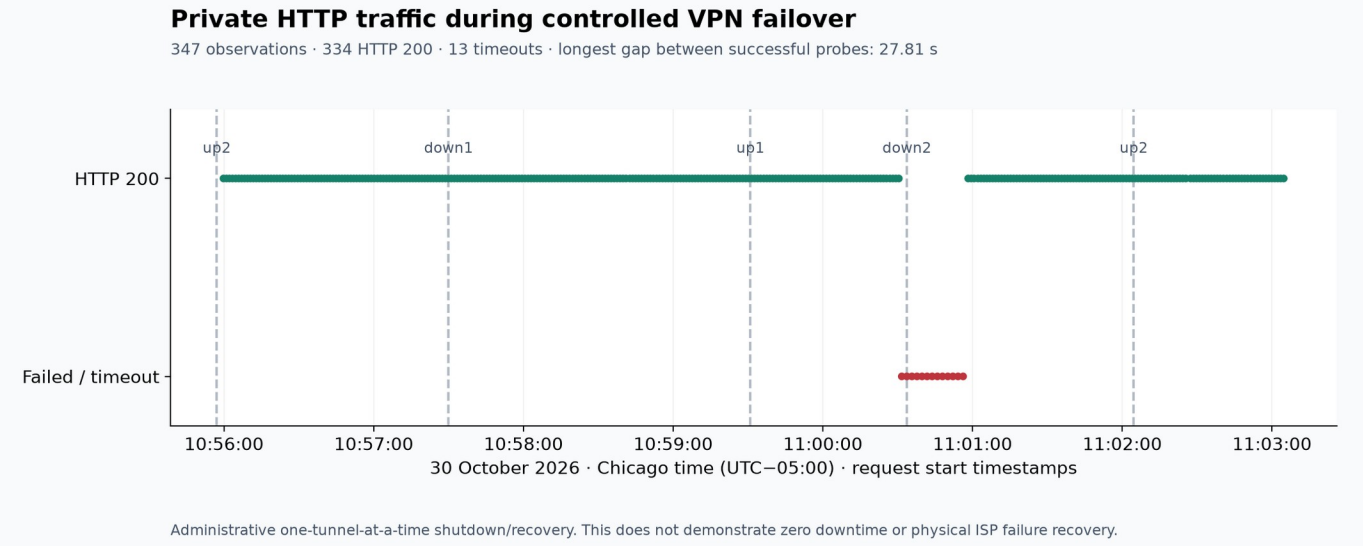
Final test administration used a temporary Systems Manager role. EC2 SSH ingress was closed; the earlier EC2 Instance Connect SSH rule was not reopened. Subnets used the default NACL; a custom experimental NACL was not associated. The private route table remained local-only. PSKs were rotated, excluded from evidence, and retired with the deleted connection.

Troubleshooting and fixes

- Corrected GCM proposal syntax and added daemon-readiness checks after restart instead of treating a missing control socket as authentication failure.
- Traced failed application traffic despite established SAs to recurring legacy packet marks created by the strongSwan forecast plugin. Disabled the plugin and verified the conflicting rules remained absent after tunnel events.
- Handled the table-220 route interaction with a destination-specific policy rule and explicit VTI routes; preserved the home internet default route.
- Retained the first unsuccessful test and repeated validation after AWS provisioning completed; distinguished delayed telemetry from actual traffic.

Verified results and lessons

Test	Observed result
Both IPsec tunnels	Authenticated IKEv2 and installed CHILD SAs; final AWS status UP
Bidirectional private traffic	Mac/Ubuntu to EC2 HTTP 200; AWS to Ubuntu proof HTTP; ICMP both directions
Each tunnel alone	Successful private traffic with only Tunnel 1 and only Tunnel 2 installed
Failover observation	334 HTTP 200 / 347 requests; 13 timeouts in one transition
Measured interruption	27.81-second gap between successful probes on transition to Tunnel 1
MTU and SSH checks	1400-byte IPv4 DF test 3/3; private TCP 22 connection blocked



Request timestamps are Chicago time. Observer phase labels can lag the control transition; all 13 failures were consecutive.

Key lessons learned

Tunnel UP is a control-plane signal, not proof of application connectivity. Check return routes, packet marks, interface counters and actual requests. Static tunnel preference can produce asymmetric traffic; forwarding must support the verified return path. Record convergence rather than claiming zero downtime.

Final lifecycle and boundaries

After testing, deleted the temporary VPN, stopped EC2, removed its temporary SSM profile/role, removed five lab-only SG rules and the AWS home-return route, and cleaned local interfaces, forwarding and project routes. Retired the temporary audit SSH key. VPC, VGW, CGW, original EC2/EBS and the separate portfolio were preserved; retained EBS remains chargeable.

This validates administrative tunnel shutdown/recovery for the designated test hosts. It does not demonstrate physical ISP failure recovery, BGP failover, private-subnet application reachability, production availability, or zero downtime.

Evidence: see the companion cloudbiz-vpn-project package for API verification, traffic records, logs, metrics, sanitized screenshots, rebuild guidance and configuration examples.

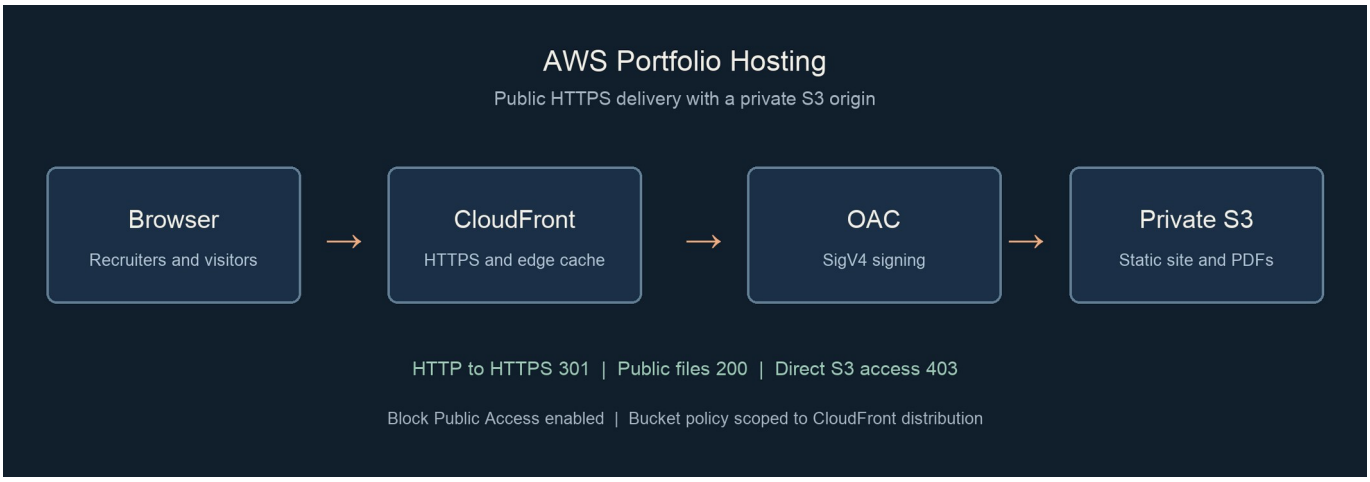
Secure AWS Portfolio Hosting

Live personal website | Amazon S3 and CloudFront | October 2026

Public portfolio

Business goal and solution

Make my engineering work, resume and sanitized project evidence accessible to recruiters through a public HTTPS website. Published a responsive static portfolio using an existing private S3 bucket and CloudFront distribution, without operating a server.



Visitors reach CloudFront; OAC signs requests to the private S3 origin. Direct unauthenticated S3 access is denied.

Implementation and security

- Audited and reused existing AWS hosting; verified all four S3 Block Public Access settings, a distribution-scoped bucket policy and OAC with always-on SigV4 signing.
- Saved the previous index and metadata for rollback, uploaded static assets before index.html, and set content types and cache controls. HTML uses no-cache; assets use a one-hour cache lifetime.
- Completed a CloudFront cache invalidation, compared public files byte-for-byte with the deployed source, and tested project dialogs, navigation and document downloads.

Verified results

Public site files returned HTTP 200 and matched local source bytes. HTTP requests returned a 301 redirect to HTTPS; unauthenticated direct S3 access returned 403. CloudFront invalidation completed. Desktop and 390px mobile checks passed with no horizontal overflow.

Lessons and scope

A public site can retain a private origin. Validate origin access, cache freshness and download integrity alongside the visible page. This deployment reused existing resources and used AWS CLI; Terraform, automated CI/CD, a custom domain and Route 53 were not implemented. S3 storage/requests and CloudFront usage remain subject to AWS pricing. No uptime or load-test claim is made.

MERN DevOps Notes App

Containerization and CI project | Source and workflow review | 30 October 2026

GitHub repository

Project overview and relevance

A React/Vite notes frontend, Node.js/Express REST API, and MongoDB database packaged as a multi-service application. This project complements the VPN lab by demonstrating application builds, containers, image delivery and Kubernetes configuration. It belongs in the portfolio as a development and DevOps lab, with runtime limitations stated clearly.

Implemented components

- Dockerfiles for backend and frontend; a multi-stage frontend build serves the React output through Nginx.
- Docker Compose services, MongoDB named volume, environment configuration, and MongoDB/backend healthchecks with dependency ordering.
- GitHub Actions dependency installation, frontend build, Docker image builds, Compose configuration validation and Docker Hub publishing with SHA/latest tags.
- Kubernetes Namespace, Deployments, Services, Ingress routing, ConfigMap/Secret references, readiness/liveness probes and CPU/memory settings.
- Documentation for local manual kubectl deployment, rollout inspection, logs and troubleshooting.

Architecture

Compose path: Browser -> Nginx frontend -> Express backend -> MongoDB. The Compose backend service is named backend; the Nginx configuration currently targets backend-service and needs alignment.

Kubernetes path: Browser -> Nginx Ingress -> frontend-service or backend-service -> application Pods -> mongodb-service -> MongoDB Pod. The database Deployment currently has no persistent volume claim.

Verified CI results

Workflow run 26005521466 at commit 268f908c19e546145cd5575a091a92ab827f6441 completed successfully on 17 May 2026. Both application-build validation and Docker-image publishing jobs succeeded. The preceding two runs reviewed also succeeded.

Successful build and publish workflow

This is evidence of CI and registry delivery, not of automated cluster deployment or end-to-end application testing. Kubernetes deployment is documented as manual; no live cluster was deployed or changed during this review.

MERN review and next steps

Recommendation

Include this project now as a containerization and CI implementation, paired with the VPN lab. Before making it the main DevOps demonstration or claiming end-to-end deployment success, correct the integration and persistence issues below and publish reproducible runtime evidence.

Priority	Finding	Required improvement
High	Compose backend DNS mismatch	Use a Compose alias or environment-specific Nginx upstream; validate the full /api path.
High	Kubernetes MongoDB has no PVC	Add persistent storage and prove notes survive Pod recreation.
Medium	/health always returns 200	Separate process liveness from readiness that checks MongoDB connectivity.
Medium	CI validates builds but no app tests	Add API CRUD and proxy integration checks; retain run evidence.
Medium	MongoDB port published on host	Keep DB internal for demos or bind a necessary local port to loopback.
Medium	README status is inconsistent	Fix code fences, remove already-implemented items from Future Improvements, and clarify manual deployment.

Security and delivery improvements

Use immutable image tags or digests in manifests. Add input/error handling and authentication before exposing a writable notes API. Define TLS and access boundaries for a public deployment. Review Secret handling and avoid committing real credentials. These are recommendations; they were not represented as completed work in the resume.

Evidence to add next

- A local startup and CRUD smoke test through Nginx, with service health and database readiness.
- A Kubernetes rollout and recovery demonstration, including data persistence after MongoDB Pod replacement.
- A CI integration-test job and deployment evidence for any future automated CD claim.

Source references

Docker Compose service definitions

Nginx upstream configuration

Backend health endpoint

MongoDB Kubernetes Deployment